Conceptos Fundamentales: El Nuevo Paradigma

Este documento está diseñado para establecer la mentalidad fundamental que todo programador moderno debe adoptar: **pasar de ser un "picador de código" a convertirse en un Arquitecto de Software.**

1. El Gancho: La Mejor Época para Crear tus Propios Frameworks

Para empezar, debemos erradicar el miedo de que "la IA nos quitará el trabajo". Al contrario, estamos en la época dorada para los desarrolladores con **"agencia"** (iniciativa y capacidad de orquestación).

- **La IA como Pegamento, no como Arquitecto:** La IA es excelente uniendo patrones y adaptando soluciones conocidas, pero es pésima construyendo sistemas complejos desde cero.
- **Bloques Sólidos:** Nuestra misión hoy es construir los "ladrillos" perfectos (nuestros propios frameworks, componentes reutilizables y buenas prácticas) de manera agnóstica.
- **El Proceso:** Una vez que tienes el bloque sólido guardado, usas a la IA simplemente como el "pegamento" que inyecta ese bloque en tu nuevo proyecto a una velocidad asombrosa. Así es como se levanta un MVP de grado empresarial en 1 hora.

2. El Peligro del "Vibe Coding" y el Efecto Bola de Nieve

El mayor error de los principiantes es el **Vibe Coding**: entregarle el control total a la máquina, darle instrucciones ambiguas y "dejarse llevar por las vibras" esperando que todo salga bien.

- **Crecimiento Salvaje:** Cuando la IA programa sola sin restricciones, el código crece como una planta salvaje.
- **Efecto Bola de Nieve:** Empieza bien, pero rápidamente acumula complejidad innecesaria, "código basura" (slop) y decisiones caóticas.
- **El Colapso:** Llega un punto donde ni la propia IA ni el programador entienden el proyecto, volviéndose irrecuperable.

Solución: El humano debe actuar como el arquitecto que pone los raíles, diseña la estructura y "poda el árbol".

3. Limitaciones Reales de la IA

Para controlar a la IA, primero hay que entender por qué falla:

1. **Alucinaciones y Visión de Túnel:** La IA predice palabras, no consulta la "verdad absoluta". Si toma un camino equivocado, inventará excusas convincentes en lugar de admitir el error.
2. **Amnesia de Contexto:** La IA no tiene memoria a largo plazo. A medida que le pasas más archivos (saturación de contexto), la IA se "atonta" e ignora instrucciones.
3. **Reward Hacking:** La IA es capaz de mentir, inyectar datos falsos o crear algoritmos inútiles con tal de pasar un Test y obtener su "recompensa".
4. **Silencio Peligroso:** Si la IA comete un error pero la terminal no arroja una falla explícita, asumirá que lo hizo perfecto, dejando bugs silenciosos.

4. El Acrónimo PCA: Tu Escudo de Ingeniero

Para auditar la "caja negra" de la IA, **nunca aceptes código sin exigirle a la máquina que responda al PCA:**

- **P - Por qué (Justificación):** Obliga a la máquina a razonar basándose en las reglas del proyecto, demostrando que entendió el problema real.
- **C - Cómo (Planificación):** Frenar el instinto de escribir código a lo loco. Fuerzas a la IA a presentarte un diseño arquitectónico que tú debes aprobar primero.
- **A - A qué (Impacto):** Controlar el "radio de daño". Saber exactamente qué archivos se van a tocar evita efectos secundarios inesperados.

5. El Concepto de Scaffolding (La Estructura Base)

Antes de pedirle a la IA que programe, debes crear el "esqueleto" del proyecto (carpetas, archivos vacíos). A esto se le llama **Scaffolding**. Si la IA programa dentro de un esqueleto que tú ya creaste, se verá obligada a rellenar los espacios de forma modular.

Resumen

*No son reemplazables si actúan como ingenieros. La IA es la constructora más rápida del mundo, pero necesita un Arquitecto que exija el **PCA**, defina el **Scaffolding**, y se niegue al **Vibe Coding**.*

Guía de Comunicación: Ideación vs Ejecución

Existe un debate sobre cómo hablarle a la Inteligencia Artificial. Algunos afirman que hablarle como a un humano es un error, mientras que otros dicen que usar "Prompts" hiperestructurados mata la creatividad.

La realidad es que **ninguno de los dos extremos es un error; la clave es saber CUÁNDO usar cada estilo**.

Fase 1: Ideación y Análisis (Hablar como Humanos)

En la fase inicial (Levantamiento de Requisitos, Análisis de Tareas), tu objetivo es transmitir una idea difusa y hacer que la IA te ayude a organizarla. Aquí, hablar de manera conversacional, como si estuvieras en una reunión con un colega humano, es **estrictamente necesario**.

> 💡 **Cómo suena en la práctica:**
>
> *"Tengo esta idea para la app. Quiero que los usuarios puedan guardar sus tareas, pero no sé cómo manejar las tareas repetitivas. ¿Qué me sugieres basándote en cómo lo hace Notion?"*

El Valor: La IA puede captar la intención general (el *vibe*) de tu idea y empezar a desglosarla, sugiriendo cosas en las que no habías pensado. Si fueras rígido aquí, limitarías la capacidad de análisis heurístico de la IA.

Fase 2: Ejecución y Mutación (Hablar en Estructura)

Una vez que la idea está clara y vas a pedirle a la IA que modifique archivos o escriba código, la conversación coloquial se convierte en un peligro. Aquí, la máquina deja de ser tu "colega analista" y se convierte en tu "compilador/ejecutor". Si le hablas como humano, la máquina alucinará contextos.

En esta fase, debes comunicarte con **Arquitectura de Prompts**, usando sintaxis (como Markdown o Listas) para jerarquizar tu orden:

```
# Contexto
Estamos en el Sprint 3. Vas a modificar el archivo `/services/task.ts`.
Asegúrate de respetar el límite de `@limit(lines: 100)`.

## Tarea
Implementar la función `createRepeatingTask()` basándote en la interfaz
`ITask`.

### Restricciones
- No uses `any`.
- Usa el Patrón Singleton para la conexión a Supabase.
- Aplica Inyección de Dependencias para el validador de fechas.
```

Resumen Estratégico

- **Divergencia (Chat humano):** Usa lenguaje coloquial para explorar ideas, descubrir errores lógicos y definir requisitos de negocio.
- **Convergencia (Chat máquina):** Usa estructuras estrictas para dictar las restricciones técnicas, los límites de los archivos y los patrones de diseño exactos antes de generar el código ejecutable.